In [6]:
```python
import pandas as pd

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
df.head()
```

Out[6]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetS
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fibe

5 rows × 21 columns

In [7]:
```python
df.shape
```

Out[7]: (7043, 21)

In [8]:
```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

In [9]:
```python
df.describe()
```

Out[9]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

In [10]:
```python
df["Churn"].value_counts()
```

Out[10]:
```
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

In [11]:
```python
df["gender"].value_counts()
```

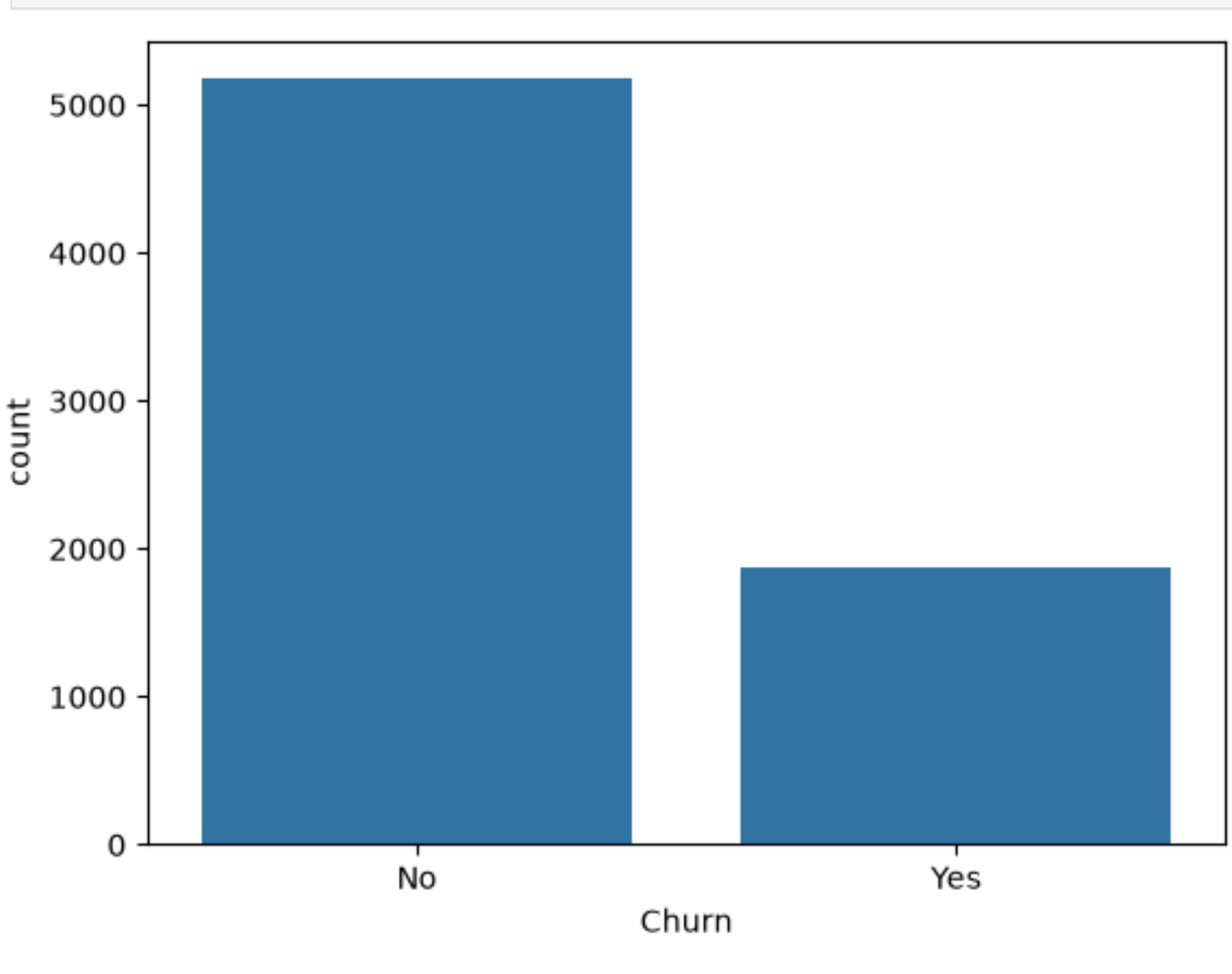
Out[11]:
```
gender
Male      3555
Female    3488
Name: count, dtype: int64
```

In [12]:
```python
df["Contract"].value_counts()
```

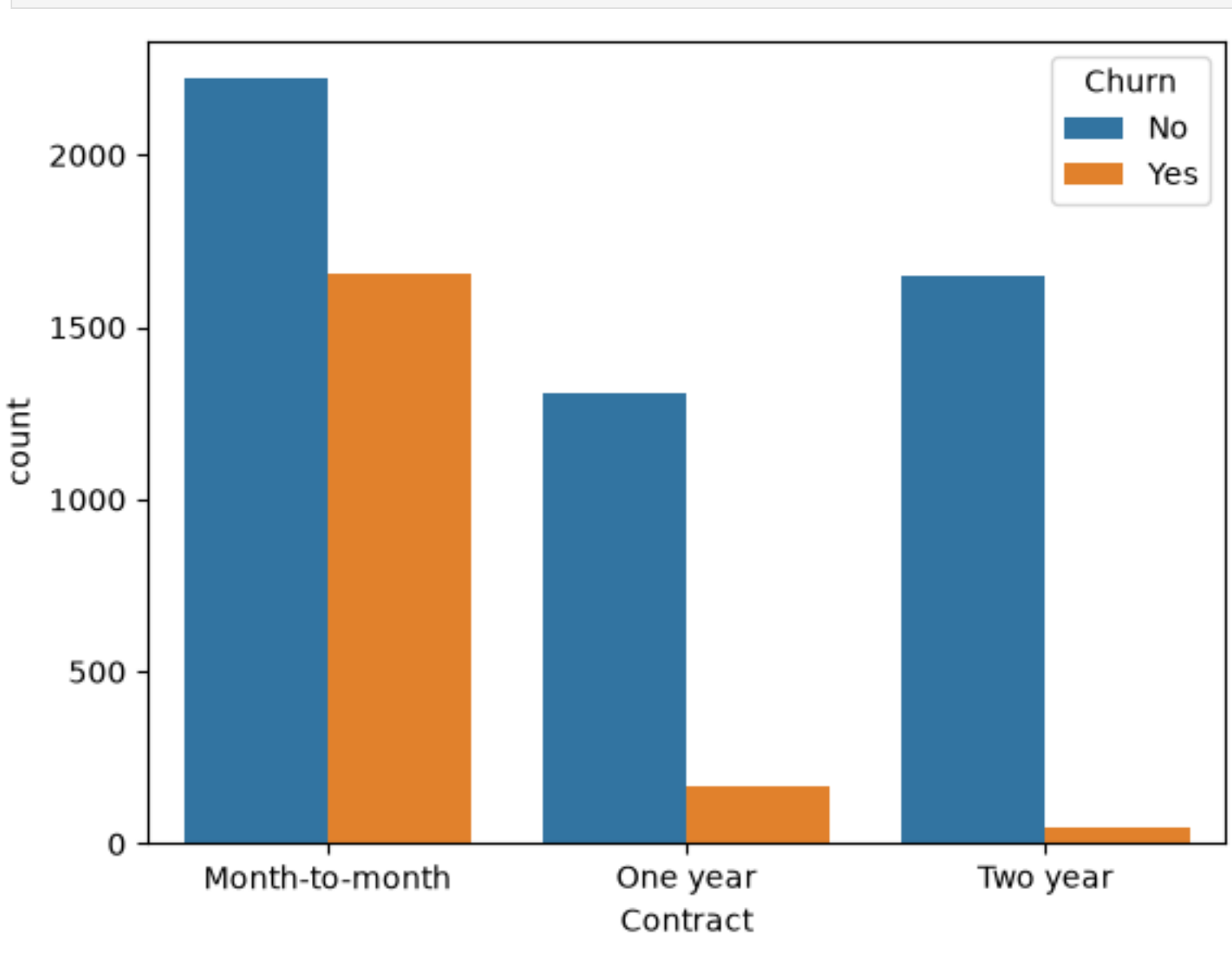
Out[12]:
```
Contract
Month-to-month    3875
Two year          1695
One year          1473
Name: count, dtype: int64
```

In [13]:
```python
import seaborn as sns
import matplotlib.pyplot as plt
```

In [14]:
```python
sns.countplot(data=df, x="Churn")
plt.show()
```



In [15]:
```python
sns.countplot(data=df, x="Contract", hue="Churn")
plt.show()
```



In [16]:
```python
pd.crosstab(df["Contract"], df["Churn"], normalize="index")
```

Out[16]:

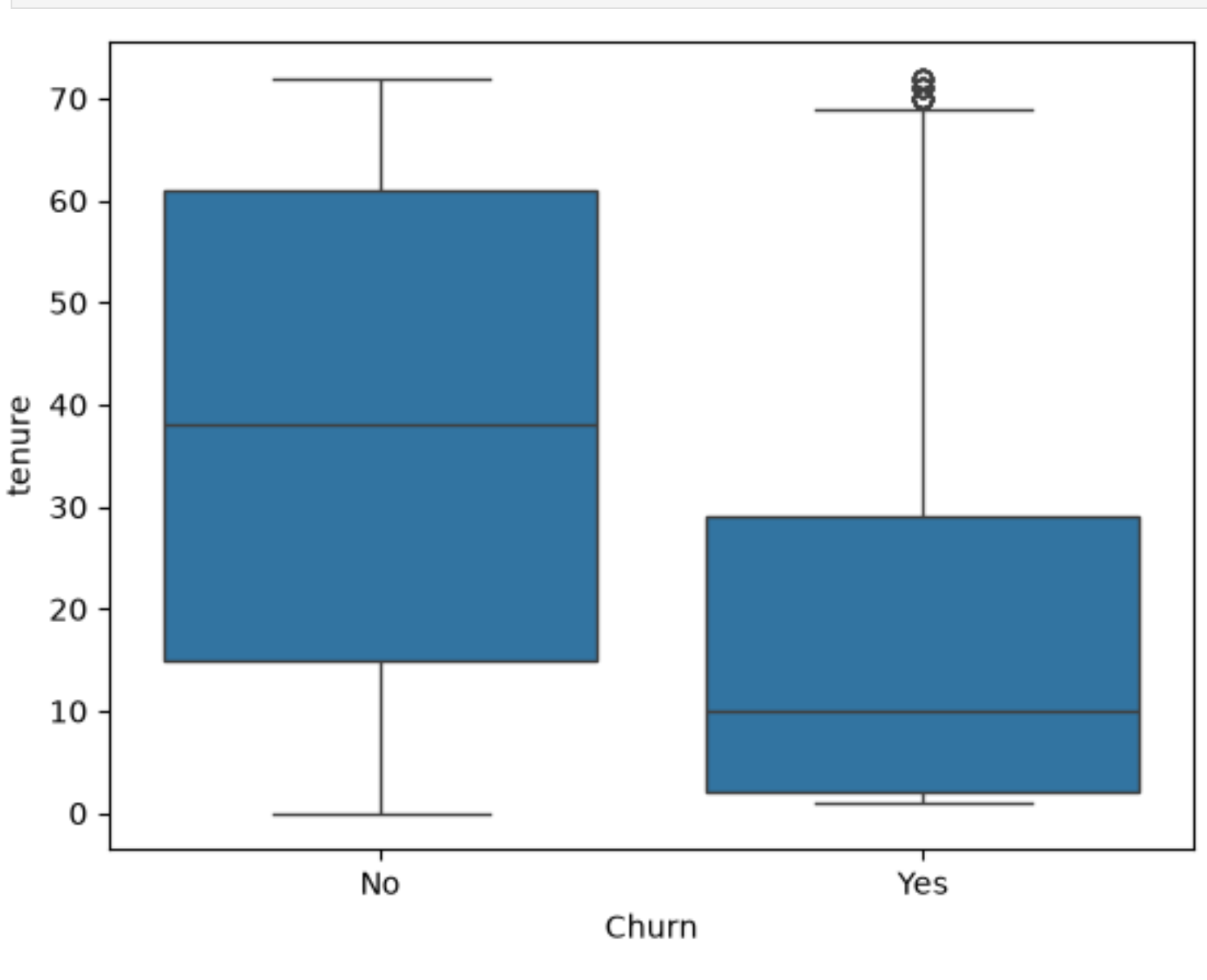
Churn	No	Yes
Contract		
Month-to-month	0.572903	0.427097
One year	0.887305	0.112695
Two year	0.971681	0.028319

In [17]:
```python
pd.crosstab(
    df["Contract"],
    df["Churn"],
    normalize="index"
) * 100
```

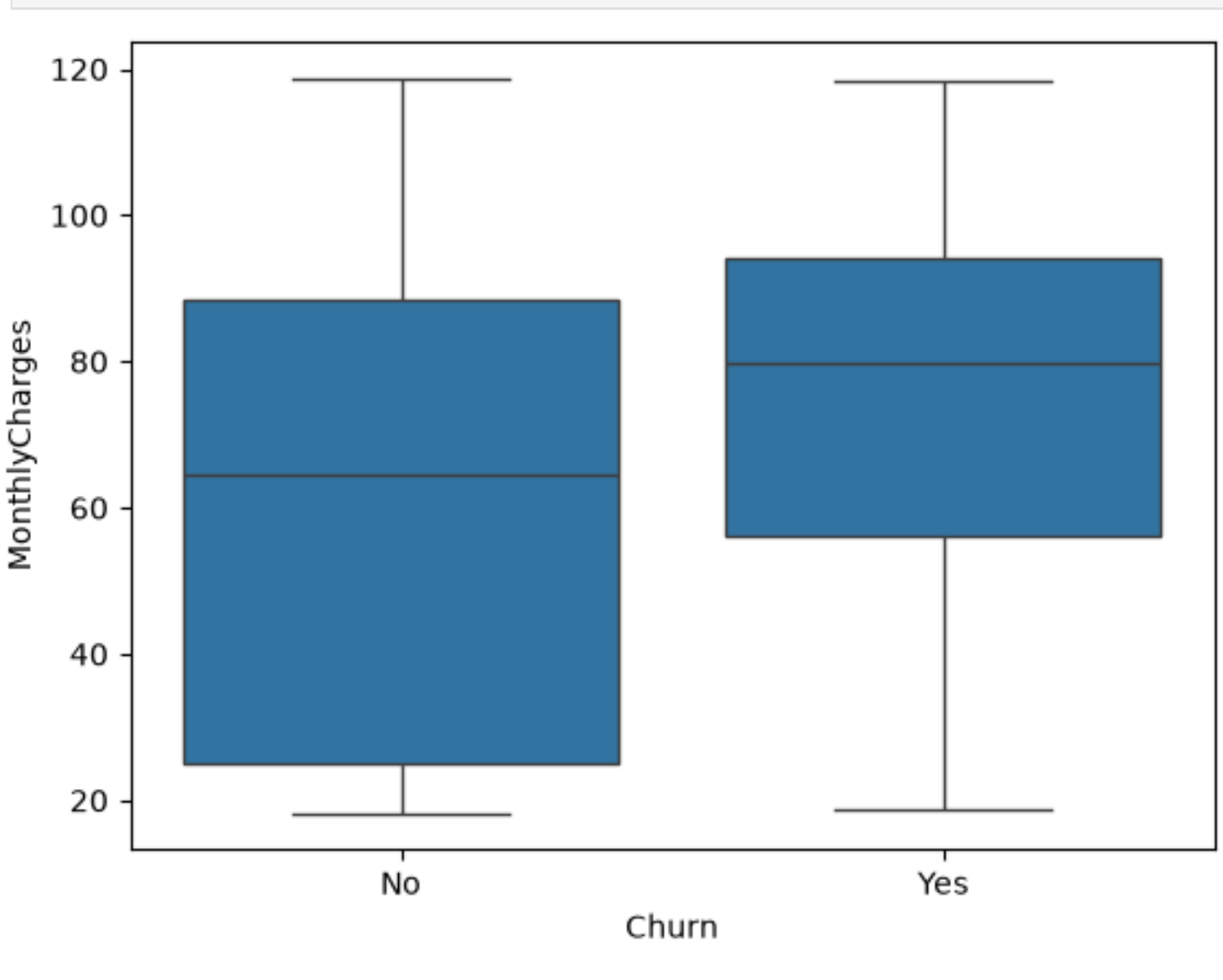
Out[17]:

Churn	No	Yes
Contract		
Month-to-month	57.290323	42.709677
One year	88.730482	11.269518
Two year	97.168142	2.831858

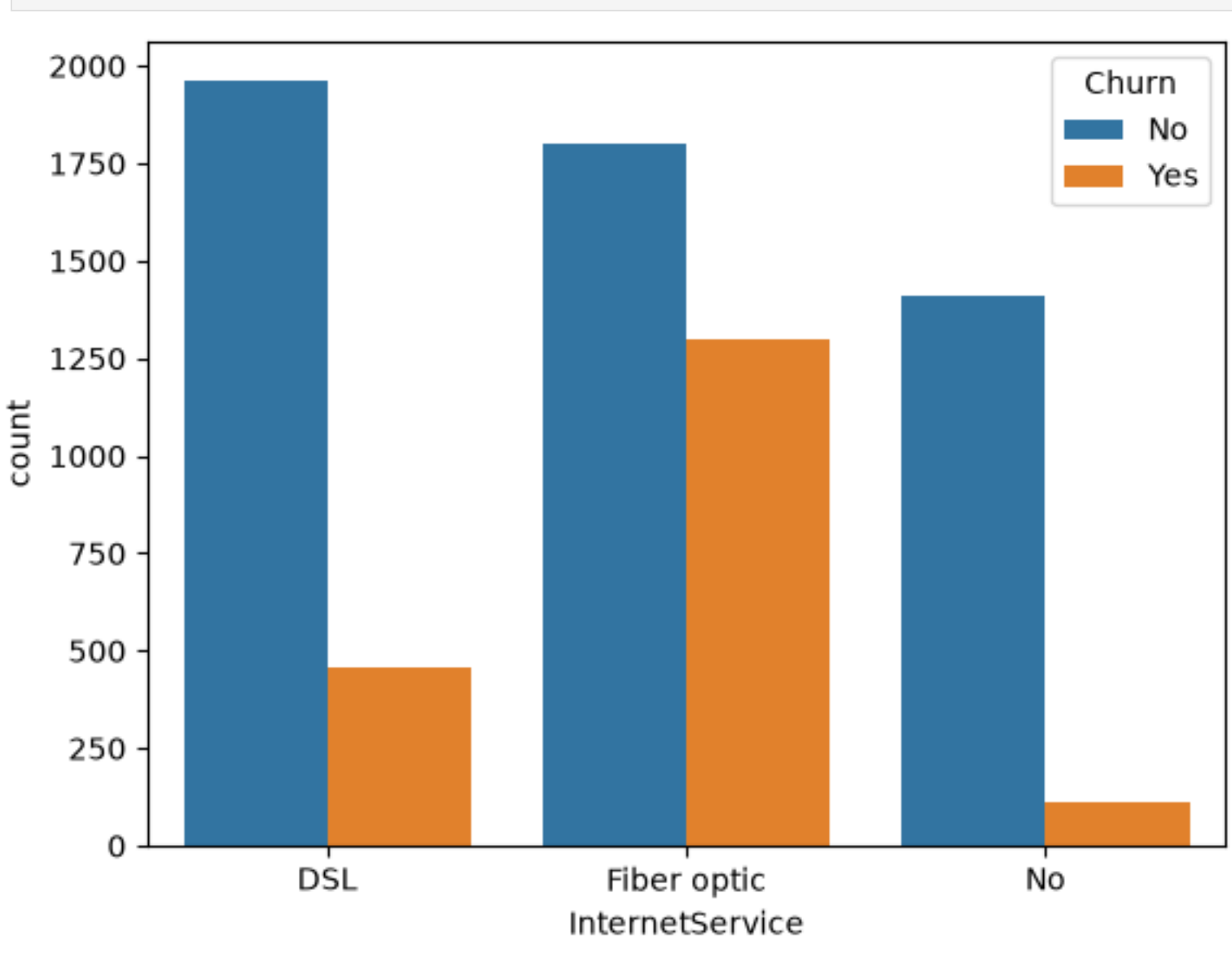
In [18]:
```python
sns.boxplot(data=df, x="Churn", y="tenure")
plt.show()
```



In [19]:
```python
sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
plt.show()
```



In [20]:
```python
sns.countplot(
    data=df,
    x="InternetService",
    hue="Churn"
)
plt.show()
```



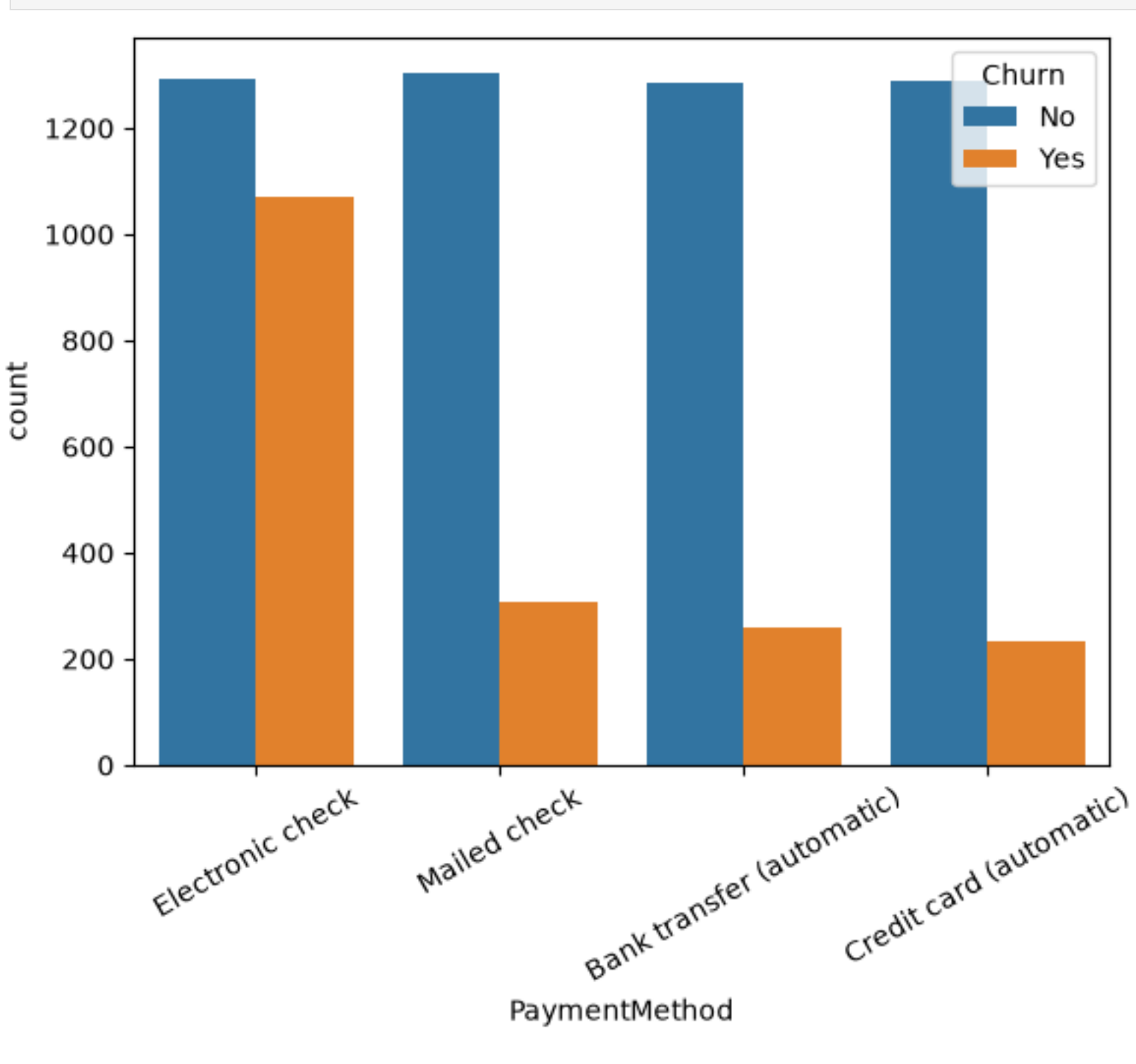
In [21]:
```python
pd.crosstab(
    df["InternetService"],
    df["Churn"],
    normalize="index"
) * 100
```

Out[21]:

Churn	No	Yes
InternetService		
DSL	81.040892	18.959108
Fiber optic	58.107235	41.892765
No	92.595020	7.404980

In [22]:
```python
sns.countplot(
    data=df,
    x="PaymentMethod",
    hue="Churn"
)

plt.xticks(rotation=30)
plt.show()
```



In [23]:
```python
pd.crosstab(
    df["PaymentMethod"],
    df["Churn"],
    normalize="index"
) * 100
```

Out[23]:

Churn	No	Yes
PaymentMethod		
Bank transfer (automatic)	83.290155	16.709845
Credit card (automatic)	84.756899	15.243101
Electronic check	54.714588	45.285412
Mailed check	80.893300	19.106700

In [24]:
```python
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"],
    errors="coerce"
)
```

In [25]:
```python
df["TotalCharges"].dtype
```

Out[25]: dtype('float64')

In [26]:
```python
df["TotalCharges"].isnull().sum()
```

Out[26]: np.int64(11)

In [27]:
```python
df[df["TotalCharges"].isnull()]
```

Out[27]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Interr
488	4472-LVYGI	Female	0	Yes	Yes	0	No	No phone service	
753	3115-CZMZD	Male	0	No	Yes	0	Yes	No	
936	5709-LVOEQ	Female	0	Yes	Yes	0	Yes	No	
1082	4367-NUYAO	Male	0	Yes	Yes	0	Yes	Yes	
1340	1371-DWPAZ	Female	0	Yes	Yes	0	No	No phone service	
3331	7644-OMVMY	Male	0	Yes	Yes	0	Yes	No	
3826	3213-VVOLG	Male	0	Yes	Yes	0	Yes	Yes	
4380	2520-SGTTA	Female	0	Yes	Yes	0	Yes	No	
5218	2923-ARZLG	Male	0	Yes	Yes	0	Yes	No	
6670	4075-WKNIU	Female	0	Yes	Yes	0	Yes	Yes	
6754	2775-SEFEE	Male	0	No	Yes	0	Yes	Yes	

11 rows × 21 columns

In [28]:
```python
df["TotalCharges"] = df["TotalCharges"].fillna(0)
```

In [29]:
```python
df["TotalCharges"].isnull().sum()
```

Out[29]: np.int64(0)

In [30]:
```python
df["Churn"].value_counts()
```

Out[30]:
```
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

In [31]:
```python
df["Churn"] = df["Churn"].map({
    "No": 0,
    "Yes": 1
})
```

In [32]:
```python
df["Churn"].value_counts()
```

Out[32]:
```
Churn
0    5174
1    1869
Name: count, dtype: int64
```

In [33]:
```python
df = pd.get_dummies(df, drop_first=True)
```

In [34]:
```python
df.head()
```

Out[34]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn	customerID_0003-MKNFE	customerID_0004-TLHLJ	cu
0	0	1	29.85	29.85	0	False	False	
1	0	34	56.95	1889.50	0	False	False	
2	0	2	53.85	108.15	1	False	False	
3	0	45	42.30	1840.75	0	False	False	
4	0	2	70.70	151.65	1	False	False	

5 rows × 7073 columns

In [35]:
```python
df.shape
```

Out[35]: (7043, 7073)

In [36]:
```python
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

In [37]:
```python
from sklearn.model_selection import train_test_split
```

In [38]:
```python
df = pd.get_dummies(df, drop_first=True)
```

In [39]:
```python
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

In [40]:
```python
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

df["TotalCharges"] = df["TotalCharges"].fillna(0)

df["Churn"] = df["Churn"].map({
    "No": 0,
    "Yes": 1
})
```

In [41]:
```python
df = df.drop("customerID", axis=1)
```

In [42]:
```python
df.columns
```

Out[42]:
```
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
       'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
       'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
       'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='str')
```

In [43]:
```python
df = pd.get_dummies(df, drop_first=True)

df.shape
```

Out[43]: (7043, 31)

In [ ]:

In [44]:
```python
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

In [45]:
```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print(X_train.shape)
print(X_test.shape)
```

```
(5634, 30)
(1409, 30)
```

In [46]:
```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=5000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print(accuracy)

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
0.8211497515968772
[[933 103]
 [149 224]]
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1036
           1       0.69      0.60      0.64       373

    accuracy                           0.82      1409
   macro avg       0.77      0.75      0.76      1409
weighted avg       0.82      0.82      0.82      1409
```